

宇宙 (universe)

将 v 排序后, 求出 v 的前缀和数组 s 。

因为在同一时刻对相同的维度进行多次操作是不优的, 所以不需要考虑同时性的问题, 只需要判断需要增加的距离之和是否足够即可。

有一个显然的性质是, 随着 k 递增, 需要考虑到的维度 i 是单调不降的且满足 $i > k$, 于是可以双指针维护 i , 如果 $v_{i+1} \times i - s_i \leq k \times v_{i+1}$ 或 $i \leq k$ 就将 i 增加 1。

显然答案满足 $ans \times k \geq ans \times i - s_i$, 可以得到答案即为 $\left\lceil \frac{s_i}{i - k} \right\rceil$ 。

时间复杂度 $\mathcal{O}(n \log n)$, 瓶颈在于排序。

跳跃 (jump)

显然 $a_i > b_i$ 和 $a_i < b_i$ 的情况可以互相转化。不妨设 $a_i < b_i$ 。

先考虑有特殊性质 C 怎么做。显然 Yuki 可以做到不踩任何一个黑色位置，于是考虑把 n 以后的位置都看成白色，处理出每个位置 i 的后继位置 $next_i$ ，令 $next_i$ 等于 $[i + 1, i + k]$ 中最靠右的白色位置。

倍增一下，计算从 a_i 移动到 $[b_i - k, b_i - 1]$ 至少要进行多少次跳跃即可。由于保证了位置 b_i 是白色的所以这样做是正确的。

再考虑没有特殊性质 C 怎么做。考虑对于一段长度为 len 的黑色段，前面的 $len - (len \bmod k)$ 个位置是必须被经过的，且每一步的长度均为 k ，所以可以把长度为 len 的黑色段压成长度为 $len \bmod k$ 的黑色段，跑特殊性质 C 的做法后再把这 $len - (len \bmod k)$ 个位置插回去即可。具体实现可以参考 `std.cpp`。

时间复杂度 $\mathcal{O}((n + q) \log n)$ 。

圆环 (circle)

先考虑有特殊性质 B 怎么做。将所有 y_i 按照 x_i 排序, 定义 $f_{i,j}$ 表示其中一只手在 y_i 且另一只手在 j 的最小花费, 转移有两种:

- 将 y_{i-1} 处的手移动到 y_i 处: $f_{i,j} \leftarrow f_{i-1,j} + \text{dis}(y_{i-1}, y_i);$
- 将另一只手移动到 y_i 处: $f_{i,y_{i-1}} \leftarrow \min \left(\min_{j=1}^m (f_{i-1,j} + \text{dis}(j, y_i)), f_{i,y_{i-1}} \right);$

其中 $\text{dis}(a, b)$ 表示 a, b 之间的距离。

接下来考虑怎么快速转移。

我们可以把 $\text{dis}(a, b)$ 拆开, 分 4 类讨论。那么现在的形式就变得好看了很多, 我们只需要压掉 f 的第一维, 上线段树优化即可。

再考虑没有特殊性质 B 怎么做。对于两个音符同时出现的情况, 我们可以对它们钦定一个先后顺序, 只需要强制第二个音符不进行第一种转移即可。

时间复杂度 $\mathcal{O}(n \log m)$ 。

翻转 (reverse)

考虑从两边往中间填。

假设填好之后变成 $\overline{a_1 a_2 \cdots b_2 b_1} \times c = \overline{b_1 b_2 \cdots a_2 a_1}$ 。

对于二元组 (a_i, b_i) ，可以计算出这两位进位和被进位的值，让 (a_i, b_i) 向合法的 (a_{i+1}, b_{i+1}) 连边，如果 c 合法会连成自动机。

起点 (a_i, b_i) 是不需要 b_{i-1} 进位， a_i 不进位且不为 0 的点。

终点的要求是：

1. 若计算奇数位的数，则终点 (a_i, b_i) 需要满足 $a_i = b_i$ 。
2. 若计算偶数位的数，则终点 (a_i, b_i) 的进位值要自洽。

时间复杂度 $\mathcal{O}(\text{poly}(k) \log_k n)$ 。